

1. Run process-run.py with the following flags: -l 5:100,5:100. What should the CPU utilization be (e.g., the percent of time the CPU is in use?)
Why do you know this? Use the -c and -p flags to see if you were right.

```
zhiyuan@CS5600:~/ostep-homework/cpu-intro$ ./process-run.py -l 5:100,5:100
Produce a trace of what would happen when you run these processes:
Process 0
  cpu
  cpu
  cpu
  cpu
  cpu

Process 1
  cpu
  cpu
  cpu
  cpu
  cpu

Important behaviors:
  System will switch when the current process is FINISHED or ISSUES AN IO
  After IOs, the process issuing the IO will run LATER (when it is its turn)
```

Should be 100%, cause p1 and p0 are always using cpu, no process is blocked.

2. Now run with these flags: ./process-run.py -l 4:100,1:0. These flags specify one process with 4 instructions (all to use the CPU), and one that simply issues an I/O and waits for it to be done. How long does it take to complete both processes? Use -c and -p to find out if you were right.

```
zhiyuan@CS5600:~/ostep-homework/cpu-intro$ ./process-run.py -l 4:100,1:0
Produce a trace of what would happen when you run these processes:
Process 0
  cpu
  cpu
  cpu
  cpu

Process 1
  io
  io_done

Important behaviors:
  System will switch when the current process is FINISHED or ISSUES AN IO
  After IOs, the process issuing the IO will run LATER (when it is its turn)
```

11 ticks, p0 using cpu for 4 ticks, p1 using cpu in io for 1 tick then blocked for 5 ticks then be done using cpu for 1 tick, $4+1+5+1=11$ ticks.

3. Switch the order of the processes: -l 1:0,4:100. What happens now? Does switching the order matter? Why? (As always, use -c and -p to see if you were right)

```

zhiyuan@CS5600:~/ostep-homework/cpu-intro$ ./process-run.py -l 1:0,4:100
Produce a trace of what would happen when you run these processes:
Process 0
  io
  io_done
Process 1
  cpu
  cpu
  cpu
  cpu

Important behaviors:
  System will switch when the current process is FINISHED or ISSUES AN IO
  After IOs, the process issuing the IO will run LATER (when it is its turn)

```

Should be 7 ticks, as p0 run first, it will use CPU for first tick then p1 start using CPU for 4 ticks, after io p0 using CPU for 1 tick.

4. We'll now explore some of the other flags. One important flag is `-S`, which determines how the system reacts when a process issues an I/O. With the flag set to `SWITCH ON END`, the system will NOT switch to another process while one is doing I/O, instead waiting until the process is completely finished. What happens when you run the following two processes (`-l 1:0,4:100 -c -S SWITCH ON END`), one doing I/O and the other doing CPU work?

```

zhiyuan@CS5600:~/ostep-homework/cpu-intro$ ./process-run.py -l 1:0,4:100 -c -S SWITCH_ON_END
Time      PID: 0      PID: 1      CPU      IOs
1         RUN:io    READY      1
2         BLOCKED  READY      1
3         BLOCKED  READY      1
4         BLOCKED  READY      1
5         BLOCKED  READY      1
6         BLOCKED  READY      1
7*        RUN:io_done  READY      1
8         DONE     RUN:cpu     1
9         DONE     RUN:cpu     1
10        DONE     RUN:cpu     1
11        DONE     RUN:cpu     1

```

Same as question 2, cause p1 will not using CPU until p0 is done.

5. Now, run the same processes, but with the switching behavior set to switch to another process whenever one is WAITING for I/O (`-l 1:0,4:100 -c -S SWITCH ON IO`). What happens now? Use `-c` and `-p` to confirm that you are right.

Same as question3, it will use 7 ticks.

```

zhiyuan@CS5600:~/ostep-homework/cpu-intro$ ./process-run.py -l 1:0,4:100 -c -p -S SWITCH_ON_IO
Time      PID: 0      PID: 1      CPU      IOs
1         RUN:io      READY      1         1
2         BLOCKED    RUN:cpu     1         1
3         BLOCKED    RUN:cpu     1         1
4         BLOCKED    RUN:cpu     1         1
5         BLOCKED    RUN:cpu     1         1
6         BLOCKED    DONE        1         1
7*        RUN:io_done  DONE        1         1

Stats: Total Time 7
Stats: CPU Busy 6 (85.71%)
Stats: IO Busy 5 (71.43%)

```

6. One other important behavior is what to do when an I/O completes. With `-I IO RUN LATER`, when an I/O completes, the process that issued it is not necessarily run right away; rather, whatever was running at the time keeps running. What happens when you run this combination of processes? (`./process-run.py -l 3:0,5:100,5:100,5:100 -S SWITCH ON IO -c -p -I IO RUN LATER`) Are system resources being effectively utilized?

```

zhiyuan@CS5600:~/ostep-homework/cpu-intro$ ./process-run.py -l 3:0,5:100,5:100,5:100 -S SWITCH_ON_IO -I IO_RUN_LATER -c -p
Time      PID: 0      PID: 1      PID: 2      PID: 3      CPU      IOs
1         RUN:io      READY      READY      READY      1         1
2         BLOCKED    RUN:cpu     READY      READY      1         1
3         BLOCKED    RUN:cpu     READY      READY      1         1
4         BLOCKED    RUN:cpu     READY      READY      1         1
5         BLOCKED    RUN:cpu     READY      READY      1         1
6         BLOCKED    RUN:cpu     READY      READY      1         1
7*        READY      DONE        RUN:cpu     READY      1         1
8         READY      DONE        RUN:cpu     READY      1         1
9         READY      DONE        RUN:cpu     READY      1         1
10        READY      DONE        RUN:cpu     READY      1         1
11        READY      DONE        RUN:cpu     READY      1         1
12        READY      DONE        DONE        RUN:cpu     1         1
13        READY      DONE        DONE        RUN:cpu     1         1
14        READY      DONE        DONE        RUN:cpu     1         1
15        READY      DONE        DONE        RUN:cpu     1         1
16        READY      DONE        DONE        RUN:cpu     1         1
17        RUN:io_done  DONE        DONE        DONE        1         1
18        RUN:io      DONE        DONE        DONE        1         1
19        BLOCKED    DONE        DONE        DONE        1         1
20        BLOCKED    DONE        DONE        DONE        1         1
21        BLOCKED    DONE        DONE        DONE        1         1
22        BLOCKED    DONE        DONE        DONE        1         1
23        BLOCKED    DONE        DONE        DONE        1         1
24*       RUN:io_done  DONE        DONE        DONE        1         1
25        RUN:io      DONE        DONE        DONE        1         1
26        BLOCKED    DONE        DONE        DONE        1         1
27        BLOCKED    DONE        DONE        DONE        1         1
28        BLOCKED    DONE        DONE        DONE        1         1
29        BLOCKED    DONE        DONE        DONE        1         1
30        BLOCKED    DONE        DONE        DONE        1         1
31*       RUN:io_done  DONE        DONE        DONE        1         1

Stats: Total Time 31
Stats: CPU Busy 21 (67.74%)
Stats: IO Busy 15 (48.39%)

```

P0 will wait for CPU so it will be in io for 3 times and wait CPU for 10 ticks so it will takes 31 ticks.

7. Now run the same processes, but with `-I IO RUN IMMEDIATE` set, which immediately runs the process that issued the I/O. How does this behavior differ? Why might running a process that just completed an I/O again be a good idea?

Compare the output with question 6, p0 will take over CPU immediately so that other process can using CPU while p0 is BLOCKED, which help save 10

ticks of CPU time. As CPU is always be using.

```
zhiyuan@CS5600:~/ostep-homework/cpu-intro$ ./process-run.py -l 3:0,5:100,5:100,5:100 -S SWITCH_ON_IO -I IO_RUN_IMMEDIATE -c -p
Time   PID: 0      PID: 1      PID: 2      PID: 3      CPU      IOs
1      RUN:io      READY      READY      READY      1
2      BLOCKED    RUN:cpu     READY      READY      1      1
3      BLOCKED    RUN:cpu     READY      READY      1      1
4      BLOCKED    RUN:cpu     READY      READY      1      1
5      BLOCKED    RUN:cpu     READY      READY      1      1
6      BLOCKED    RUN:cpu     READY      READY      1      1
7*     RUN:io_done DONE      READY      READY      1
8      RUN:io      DONE      READY      READY      1
9      BLOCKED    DONE      RUN:cpu     READY      1      1
10     BLOCKED    DONE      RUN:cpu     READY      1      1
11     BLOCKED    DONE      RUN:cpu     READY      1      1
12     BLOCKED    DONE      RUN:cpu     READY      1      1
13     BLOCKED    DONE      RUN:cpu     READY      1      1
14*    RUN:io_done DONE      DONE      READY      1
15     RUN:io      DONE      DONE      READY      1
16     BLOCKED    DONE      DONE      RUN:cpu     1      1
17     BLOCKED    DONE      DONE      RUN:cpu     1      1
18     BLOCKED    DONE      DONE      RUN:cpu     1      1
19     BLOCKED    DONE      DONE      RUN:cpu     1      1
20     BLOCKED    DONE      DONE      RUN:cpu     1      1
21*    RUN:io_done DONE      DONE      DONE      1

Stats: Total Time 21
Stats: CPU Busy 21 (100.00%)
Stats: IO Busy 15 (71.43%)
```

8. Now run with some randomly generated processes using flags `-s 1 -l 3:50,3:50` or `-s 2 -l 3:50,3:50` or `-s 3 -l 3:50, 3:50`. See if you can predict how the trace will turn out. What happens when you use the flag `-I IO RUN IMMEDIATE` versus that flag `-I IO RUN LATER`? What happens when you use the flag `-S SWITCH ON IO` versus `-S SWITCH ON END`?

```
zhiyuan@CS5600:~/ostep-homework/cpu-intro$ ./process-run.py -s 1 -l 3:50,3:50
./process-run.py -s 2 -l 3:50,3:50
./process-run.py -s 3 -l 3:50,3:50
Produce a trace of what would happen when you run these processes:
Process 0
  cpu
  io
  io_done
  io
  io_done

Process 1
  cpu
  cpu
  cpu

Important behaviors:
  System will switch when the current process is FINISHED or ISSUES AN IO
  After IOs, the process issuing the IO will run LATER (when it is its turn)
```

```
hiyuan@CS5600:~/ostep-homework/cpu-intro$ ./process-run.py -s 1 -l
3:50,3:50 -S SWITCH_ON_IO -I IO_RUN_LATER -c -p

./process-run.py -s 1 -l 3:50,3:50 -S SWITCH_ON_IO -I IO_RUN_IMMEDIATE -c
-p

./process-run.py -s 1 -l 3:50,3:50 -S SWITCH_ON_END -I IO_RUN_LATER -c
-p
```

```
./process-run.py -s 1 -l 3:50,3:50 -S SWITCH_ON_END -I IO_RUN_IMMEDIATE -c
-p
```

Time	PID: 0	PID: 1	CPU	IOs
1	RUN:cpu	READY	1	
2	RUN:io	READY	1	
3	BLOCKED	RUN:cpu	1	1
4	BLOCKED	RUN:cpu	1	1
5	BLOCKED	RUN:cpu	1	1
6	BLOCKED	DONE		1
7	BLOCKED	DONE		1
8*	RUN:io_done	DONE	1	
9	RUN:io	DONE	1	
10	BLOCKED	DONE		1
11	BLOCKED	DONE		1
12	BLOCKED	DONE		1
13	BLOCKED	DONE		1
14	BLOCKED	DONE		1
15*	RUN:io_done	DONE	1	

Stats: Total Time 15

Stats: CPU Busy 8 (53.33%)

Stats: IO Busy 10 (66.67%)

Time	PID: 0	PID: 1	CPU	IOs
1	RUN:cpu	READY	1	
2	RUN:io	READY	1	
3	BLOCKED	RUN:cpu	1	1

4	BLOCKED	RUN:cpu	1	1
5	BLOCKED	RUN:cpu	1	1
6	BLOCKED	DONE		1
7	BLOCKED	DONE		1
8*	RUN:io_done	DONE	1	
9	RUN:io	DONE	1	
10	BLOCKED	DONE		1
11	BLOCKED	DONE		1
12	BLOCKED	DONE		1
13	BLOCKED	DONE		1
14	BLOCKED	DONE		1
15*	RUN:io_done	DONE	1	

Stats: Total Time 15

Stats: CPU Busy 8 (53.33%)

Stats: IO Busy 10 (66.67%)

Time	PID: 0	PID: 1	CPU	IOs
1	RUN:cpu	READY	1	
2	RUN:io	READY	1	
3	BLOCKED	READY		1
4	BLOCKED	READY		1
5	BLOCKED	READY		1
6	BLOCKED	READY		1
7	BLOCKED	READY		1
8*	RUN:io_done	READY	1	
9	RUN:io	READY	1	

10	BLOCKED	READY		1
11	BLOCKED	READY		1
12	BLOCKED	READY		1
13	BLOCKED	READY		1
14	BLOCKED	READY		1
15*	RUN:io_done	READY	1	
16	DONE	RUN:cpu	1	
17	DONE	RUN:cpu	1	
18	DONE	RUN:cpu	1	

Stats: Total Time 18

Stats: CPU Busy 8 (44.44%)

Stats: IO Busy 10 (55.56%)

Time	PID: 0	PID: 1	CPU	IOs
1	RUN:cpu	READY	1	
2	RUN:io	READY	1	
3	BLOCKED	READY		1
4	BLOCKED	READY		1
5	BLOCKED	READY		1
6	BLOCKED	READY		1
7	BLOCKED	READY		1
8*	RUN:io_done	READY	1	
9	RUN:io	READY	1	
10	BLOCKED	READY		1
11	BLOCKED	READY		1
12	BLOCKED	READY		1

13	BLOCKED	READY		1
14	BLOCKED	READY		1
15*	RUN:io_done	READY	1	
16	DONE	RUN:cpu	1	
17	DONE	RUN:cpu	1	
18	DONE	RUN:cpu	1	

Stats: Total Time 18

Stats: CPU Busy 8 (44.44%)

Stats: IO Busy 10 (55.56%)